

ufpipe — Ulsan-Fault catalog pipeline

Reference manual: detection → association → augment → PHS → HYPOINVERSE

Long-term (2010–present) earthquake catalog for the Ulsan Fault, SE Korea

package `src/ufpipe`

This manual documents the `ufpipe` package (`src/ufpipe`) — the automated pipeline that turns continuous waveforms into a located earthquake catalog. The related *relocation* / *picker-comparison* pipeline (`detection_test/`) has its own guide: `detection_test/reloc_2016_uf/PIPELINE.md` + `study_guide.pdf`.

Contents

1	What the pipeline does	1
1.1	Two orthogonal “model” dimensions	1
2	Install & environment	2
3	The five stages	2
3.1	Source of truth for pick↔event assignment	3
4	Parameters (<code>src/ufpipe/config.py</code> — the single disclosed source)	3
4.1	Detection / DSP	3
4.2	Association (PyOcto) — REGION (lenient) vs REGION_STRICT	3
4.3	Velocity model & location	4
5	How to run	4
6	Outputs & layout	4
7	Robustness & invariants	5
8	Troubleshooting / provenance checks	6

1 What the pipeline does

Build an **internally consistent, long-term earthquake catalog** for the Ulsan Fault region (SE Korea), 2010–present, from continuous seismic waveforms, using AI phase pickers so small events are detected uniformly even as the recording network grows. Every stage is scripted and parameterized; results are reproducible.

Detection	Association	Location	Catalog
PhaseNet / PhaseNet+	PyOcto (+augment)	HYPOINVERSE	located events
<i>picks per station-day</i> →	<i>events + assignments</i> →	<i>PHS</i> → <i>.sum/.arc</i> →	<i>UF{year}.sum</i>

1.1 Two orthogonal “model” dimensions

Dimension	Flag	Values
Picker model	<code>--model</code>	stead, original, phasenet_plus, eqt, ...
Velocity model	<code>--velmodel</code>	kim1983, kim2011

Any picker model can be located with any velocity model. `stead/original` are SeisBench PhaseNet weights; `phasenet_plus` is EQNet PhaseNet+ (needs a local EQNet clone).

2 Install & environment

One-time setup

1. `conda env create -f environment.yml && conda activate ulsan` (the code also runs in base here)
2. `pip install torch --index-url https://download.pytorch.org/whl/cu128` (match your CUDA; GPU strongly recommended)
3. `pip install -e .` (installs the `uflib` + `ufpipe` packages, editable)
4. **HYPOINVERSE**: the external `hyp1.40` binary must be on `PATH` (not pip-installable).
5. **PhaseNet+ only**: a local EQNet clone at `config.EQNET_DIR`.

After `pip install -e .`, `import ufpipes.config` and `python -m ufpipes.run_pipeline` work from any directory — no `sys.path` hacks.

3 The five stages

`STAGES = [detection, association, augment, phs, locate]` — run in order. Each is a function in `src/ufpipe/core.py`; `run_pipeline.py` chains them per year.

#	Stage	In → Out	What it does
1	detection	continuous mSEED → picks/ (per station-day)	AI picker on each station-day; anti-alias >100 Hz, resample to 100 Hz, pick P/S above threshold. GPU. Idempotent (skips done days).
2	association	picks → pyocto_*.csv + assignment	PyOcto octree associator groups picks into events (gate below). Whole-year or daily-chunked.
3	augment	assignment → augmented assignment	scan daily picks for orphans PyOcto missed, add to each event so HYPOINVERSE sees the full pick set. Backs up + overwrites in place; idempotent.
4	phs	assignment → UF{year}.phs	write the HYPOINVERSE phase file from the (augmented) assignment.
5	locate	.phs → UF{year}.{sum,arc}	run HYPOINVERSE (<code>hyp1.40</code>) with the chosen velocity model → located catalog + errors.

Pipeline order matters

augment runs *between* **association** and **phs**, so HYPOINVERSE sees the full augmented pick set and computes correct **GAP/DMIN/ERZ/RMS**. Do not reorder.

3.1 Source of truth for pick↔event assignment

The **PyOcto assignment (after augmentation)** is authoritative for which (**station**, **phase**) tuples belong to each event. Do NOT use a legacy time-window pick dump — when two events fall <60 s apart at the same station the earliest pick “wins” and leaks into the wrong event. Downstream (SAC-header writers, magnitude/QC notebooks) must read `pyocto_assignment_{velmodel}_{year}.csv`.

4 Parameters (src/ufpipe/config.py — the single disclosed source)

Change defaults here. Everything is one file.

4.1 Detection / DSP

Parameter	Value	Meaning
SAMPLING_RATE	100.0 Hz	the pickers’ trained rate; all data resampled to it
ANTIALIAS	lowpass 45 Hz, 8 corners, zerophase	applied BEFORE down-sampling (>100 Hz KG/NS)
BANDPASS	1–40 Hz, 4 corners, causal	legacy detection path; PhaseNet+ runs raw
P_THRESHOLD, S_THRESHOLD	0.2	SeisBench pick probability threshold
PNPLUS_MIN_PROB	0.3	EQNet PhaseNet+ pick threshold
PNPLUS_NT	1024×36	PhaseNet+ patch length (whole-day, <code>cut_patch</code>)
MAX_CORES	24	CPU cap (respects <code>taskset</code> affinity)

4.2 Association (PyOcto) — REGION (lenient) vs REGION_STRICT

Parameter	REGION	REGION_STRICT	Meaning
lat / lon	(34.5,37.0) / (128.5,130.0)		search area (deg)
zlim	(0, 40)		depth range (km)
time_before	300		origin-search window (s)
n_picks	4	6	min total picks
n_p_picks	2	3	min P picks
n_s_picks	2	3	min S picks
n_p_and_s_picks	1	2	min stations with both P+S (depth constraint)
pick_match_tolerance	1.5	1.0	residual cap (s) — the primary quality knob
min_node_size / _location	10/1.5	2.0/0.5	octree granularity (km)
refinement_iterations	3	8	localise+rematch cycles
n_threads	16		PyOcto threads (polite on a shared box)

Pass `--strict` to use `REGION_STRICT` (fewer, cleaner events). Velocity for association = `PYECTO_VELMODEL (kim1983)`.

4.3 Velocity model & location

`DEFAULT_VELMODEL = kim2011`; `kim1983` also available (`data/metadata/velocity_model/kim1983.csv`, a layered V_p/V_s model). `PHASE_CHANNELS = {P: HHZ, S: HHN}`.

5 How to run

Standard runs (from the repo root)

<code>python -m ufpipes.run_pipeline --model original --years 2024</code>	one year, all 5 stages
<code>python -m ufpipes.run_pipeline --model original --years 2010-2024</code>	the whole record
<code>python -m ufpipes.run_pipeline --model original --years 2015,2016 --velmodel kim1983</code>	specific years, kim1983
<code>python -m ufpipes.run_pipeline --model original --years 2024 --stage-from association</code>	resume (picks already done)
<code>python -m ufpipes.run_pipeline --model original --years 2024 --days 1-3</code>	detection on a few days (testing)
<code>python -m ufpipes.run_pipeline --model original --years 2024 --strict</code>	strict association gate

CLI flags: `--model` · `--years` ('2010-2024' or '2015,2016') · `--velmodel` · `--days` · `--device` {cuda,cpu} · `--workers` · `--strict` · `--force` (allow writing to stead) · `--stage-from` {detection,association,augment,phs,locate} (start mid-chain).

Shared 64-core server — always pin cores

Detection sizes its preprocessing pool and `torch` threads from the process CPU *affinity*, capped by `MAX_CORES=24`. Launch under `taskset` so the whole job stays within budget:

```
OMP_NUM_THREADS=1 taskset -c 0-7 python -m ufpipes.run_pipeline --model original --years 2010-2024
```

Without pinning, PhaseNet+ grabbed ~49 cores / 193 threads and starved everything. Inference is **GPU-preferred** (warns loudly, never silently falls back to CPU).

Detection is **idempotent** — it skips station-days whose picks already exist, so runs resume safely after an interruption.

6 Outputs & layout

All pipeline products go under `outputs/models/<model>/` (regenerable; gitignored):

Path	Contents
.../detection_location/<year>/picks/	per-station-day picks (stage 1)
.../pyocto/pyocto_{vm}_{year}.csv	associated events (stage 2)
.../pyocto/pyocto_assignment_{vm}_{year}.csv	pick→event assignment (authoritative)
.../station_table/stations_{year}.csv	per-year station roster (discovered from metadata)
.../HypoInv/PHS/UF{year}.phs	HYPONVERSE phase file (stage 4)
.../HypoInv/{vm}/UF{year}.{sum,arc,prt}	located catalog + errors (stage 5)

Inputs (restructured 2026-07): waveforms in `KS_KG/` (station dirs at root, parallel to `GJ/NS/`); station metadata in `data/metadata/ks_kg_station_table/`; velocity model in `data/metadata/velocity_model/`; HYPONVERSE control inputs (STA/kim*) in `data/hypoinv/`.

7 Robustness & invariants

- **Network growth is automatic.** Stations are *discovered from StationXML + on-disk coverage* each month; the deployed network for that period is what enters the run. No hard-coded station lists.
- **Mixed sampling rates handled.** >100 Hz stations (KG/NS at 200 Hz) are anti-alias-lowpassed then resampled to the pickers' 100 Hz. The anti-alias filter is applied BEFORE down-sampling (never after).
- **Velocity model, thresholds, and gates are epoch-invariant by choice** ⇒ the catalog stays internally comparable across years.
- **Idempotent & resumable** at every stage; `--stage-from` restarts mid-chain.
- **Do not edit the stead reference run** (it is the baseline). Scripts refuse `--model stead` writes unless `--force`.

8 Troubleshooting / provenance checks

Symptom	Check
Detection finds no waveforms	<code>python -c "import ufpipes.config as c,os; print(c.CONTINUOUS, os.path.isdir(c.CONTINUOUS))"</code> — must point at <code>KS_KG/</code> and exist.
<code>ImportError: ufpipes / uflib</code>	<code>pip install -e .</code> in the active env; <code>import ufpipes.config; print(__file__)</code> must be <code>src/ufpipes</code> .
Wrong pipeline imported	<code>ufpipes</code> is renamed precisely to avoid the collision with the external <code>korea-cluster-relocation pipeline</code> . Verify <code>ufpipes.config.__file__</code> is under <code>src/</code> .
Events look over/under-associated	compare <code>REGION</code> vs <code>--strict</code> ; the <code>pick_match_tolerance</code> is the primary knob.
GPU not used	inference is GPU-preferred and warns loudly; check the torch/CUDA install (never silently CPU).
Sparse early years, few events	expected — <code>KS/KG-only</code> network (2010–2016) has larger azimuthal gaps; the QC gate honestly reflects what the network resolves.

Companion docs: `docs/overview.md`, `docs/pipeline.md`, `docs/how-to-run.md`,
`docs/directory-structure.md` · `src/ufpipes/README.md` · `relocation pipeline:`
`detection_test/reloc_2016_uf/PIPELINE.md` + `study_guide.pdf`.